🔷 MERGE SORT (Algorithm in Words)

1. Start with the given array.
2. Check if the array has more than one element.
3. If yes, divide the array into two equal halves.
4. Recursively apply Merge Sort to the left half.
5. Recursively apply Merge Sort to the right half.
6. After sorting both halves, merge them into a single sorted array.
7. While merging:
    - Compare elements from both halves.
    - Place the smaller element into the final array.
    - Continue until all elements are merged.
8. The final merged array will be sorted.

🔷 QUICK SORT (Algorithm in Words)

1. Start with the given array.
2. Select a pivot element (usually the last element).
3. Rearrange the array so that:
    - All elements smaller than the pivot are placed on the left.
    - All elements greater than the pivot are placed on the right.
4. The pivot is now in its correct sorted position.
5. Recursively apply Quick Sort to the left sub-array.
6. Recursively apply Quick Sort to the right sub-array.
7. Repeat the process until the entire array is sorted.
8. 🔷 **MERGE SORT**
9. ⏱️ **Time Complexity**
10. Best Case: **O(n log n)**
11. Average Case: **O(n log n)**
12. Worst Case: **O(n log n)**

13. 👉 Same in all cases because it always divides and merges.

14. ---

15. 💾 Space Complexity

16. **O(n)**

17. 👉 Extra space is needed for temporary arrays during merging.

18. ---

19. 🔷 QUICK SORT

20. ⏱️ Time Complexity

21. Best Case: **O(n log n)**

22. Average Case: **O(n log n)**

23. Worst Case: **O(n^2)**

24. 👉 Worst case happens when pivot is smallest or largest (unbalanced partition).

25. ---

26. 💾 Space Complexity

27. Best/Average: **O(log n)**

28. Worst: **O(n)**

29. 👉 Due to recursion (function call stack).

30. ---

31. 🧠 Quick Comparison Table

32. Algorithm	**33. Best Time**	**34. Worst Time**	**35. Space**
36. Merge Sort	37. O(n log n)	38. O(n log n)	39. O(n)
40. Quick Sort	41. O(n log n)	42. O(n^2)	43. O(log n)